

Identa Mobile — Documentation

Expo / React Native client for the Identa dental-clinic system

1. Architecture & Handoff Guide

2. Feature Status

3. QA & Verification Plan

4. Continuation Roadmap

Generated 2026-06-05 · Source: src + AGENTS.md + IDENTA_WEB_REFERENCE.md · Editable markdown in `docs/`

Identa Mobile — Architecture & Handoff Guide

Audience: a developer (or agent) picking up the Identa mobile app to continue or verify it. Read this first, then `FEATURE_STATUS.md` (what's done), then `ROADMAP.md` (what's left) and `QA_VERIFICATION.md` (how to test).

Companion docs already in the repo:

- `AGENTS.md` — concise agent quick-start (stack, commands, conventions).
- `IDENTA_WEB_REFERENCE.md` — the **backend API contract & business rules**.
- `src/types/index.ts` — the source of truth for all response shapes.

Last verified against the codebase: 2026-06-05.

1. What this is

Native mobile client for **Identa**, a dental-clinic practice-management system. The mobile app is a **client only** — it talks to a separate Laravel + Sanctum backend (`.../api/v1` , prod `https://api.identa.uz/api/v1` ; Postgres + Redis + worker on Railway). The same backend powers the web app (`Identa web/identa`).

Stack (versions are load-bearing — confirm in `package.json` before assuming):

Concern	Choice
Runtime	Expo SDK <code>~54.0.33</code> , React Native <code>0.81.5</code> , React <code>19.1</code> , TypeScript <code>~5.9</code>
Architecture	New Architecture enabled (<code>app.json</code> → <code>newArchEnabled: true</code>)
Navigation	React Navigation v7 (<code>native-stack</code> + <code>bottom-tabs</code>) — not expo-router
Server state	TanStack React Query v5, persisted to AsyncStorage (offline cache)
Client state	Zustand v5 (<code>src/stores</code> : <code>auth</code> , <code>network</code> , <code>theme</code> , <code>ui</code>)
HTTP	axios (<code>src/api/client.ts</code>)
Auth storage	<code>expo-secure-store</code> (session), key <code>identa.session</code>
Crash reporting	<code>@sentry/react-native</code>
Media / native	<code>expo-image-picker</code> , <code>expo-notifications</code> , <code>expo-updates</code> (OTA), <code>expo-haptics</code>
i18n	Custom (<code>src/i18n</code>) — no i18next. Locales <code>ru</code> (default) / <code>uz</code> / <code>en</code>

Expo moves fast between SDKs. Before touching any Expo/native API, read the SDK 54 docs (`https://docs.expo.dev/versions/v54.0.0/`). Training-data answers for older SDKs are frequently wrong for 54.

2. Project structure

```
src/
  api/          axios client + per-resource modules (each has a mock branch)
  stores/       zustand: auth, network, theme, ui
  screens/      auth/ dashboard/ patients/ appointments/ payments/ settings/
  components/   feature folders (appointments, dashboard, patients, payments,
                  settings, odontogram, gallery, treatments) + ui/ primitives
  lib/          hooks + helpers (format, validation, sentry, notifications,
                  otaUpdates, offlineGuard, permissions, appointmentConflicts, ...)
  i18n/         bundled translations (ru/uz/en) + React provider
  constants/    API_URL, colors/theme, durations, locales, page size, business consts
  types/        all Api* interfaces – SOURCE OF TRUTH for response shapes
  App.tsx       React Query + persistence, providers, Sentry/OTA bootstrap
  src/navigation/index.tsx  the whole navigator graph + deep linking
```

3. Navigation graph — `src/navigation/index.tsx`

A single `<NavigationContainer>` switches on `useAuthStore(s => s.isAuthenticated)`:

```
NavigationContainer (deep-link aware)
├ NOT authenticated → AuthStack (native-stack, headers hidden)
│   ├── Login
│   ├── Register
│   ├── ForgotPassword
│   └── ResetPassword      params: { token?, email? }
└ authenticated → MainStack (native-stack)
    ├── Tabs (bottom-tabs, custom CustomTabBar)
    │   ├── Dashboard
    │   ├── Patients
    │   ├── Appointments
    │   └── Payments
    ├── Settings          presentation: modal
    ├── PatientDetail     params: { id }
    └ PatientOdontogram   params: { patientId, patientName? }
```

- **Two global bottom-sheets** are rendered as siblings of the stack and driven by

`useUIStore`: `AppointmentCreateSheet` and `PatientFormSheet`. They invalidate the relevant query keys (`appointments` / `dashboard` / `patients`) on success. This is why "create patient" / "create appointment" work from multiple screens.

- On mount, `MainNavigator` registers the Expo push token

(`getExpoPushToken()` → `registerDeviceToken()`); failure is silent.

- **Deep linking** (`linking`): prefixes `identa://` and `https://identa.uz`.

Routes: `home` / `patients` / `appointments` / `payments`, `settings`, `patient/:id`, `patient/:patientId/odontogram`, `reset-password` (resolves while logged out). ⚠️ `app.json` also declares `app.identa.uz` in `associatedDomains` / `intent filters`, but the `linking.prefixes` list only includes `identa.uz` — keep them in sync if universal links on `app.identa.uz` are required.

4. API layer — `src/api/`

`client.ts` (the part most likely to trip you up)

- axios instance: `baseURL = API_URL` (see §6), `withCredentials: true` (needed for

the CSRF cookie jar), `timeout: 20s`.

- **Request interceptor** lazily `require()`s the auth store and attaches

Authorization: Bearer <access> (skippable per-request via `_skipAuthHeader`).

- `ApiError` normalizes every failure to a `kind` :

`network` | `timeout` | `unauthorized` | `forbidden` | `not_found` | `validation` | `server` | `unknown` plus `status` , `fieldErrors` (422), `cause` .

Always branch on `error.kind` , never on raw axios shapes.

- `401` → **refresh flow**: a single-flight `refreshInFlight` coalesces concurrent

`401`s, calls `POST /auth/refresh` with the `refresh_token` (bare axios, no bearer), on success `setTokens` + retries the original request once (`_retriedAfterRefresh` guard); on failure → `logout()` . A `403` with body `code: 'account_inactive'` also forces `logout()` . `server` / `unknown` kinds are reported to Sentry.

`csrf.ts` (Laravel `web` -middleware routes only)

Login, register, forgot-password, reset-password, change-password run through the Laravel `web` group and need a **CSRF bootstrap**: `GET /auth/csrf-token` → `{ token }` + `identa-session` cookie, then echo `X-CSRF-TOKEN` + `Cookie` on the POST. Memoized for 10 min, single-flight. `/auth/refresh` **does NOT need CSRF** (it's outside the `web` group). See `IDENTA_WEB_REFERENCE.md §2` for the contract.

Per-resource modules + mock toggles

14 slice files (`auth`, `patients`, `appointments`, `dashboard`, `payments`, `treatments`, `odontogram`, `profile`, `team`, `sessions`, `devices`, `notifications`, ...). Each mutating call gates on `requireOnline()` (`lib/offlineGuard.ts`).

Every slice ships an inline **mock branch** so the app runs with no backend:

- Master switch `EXPO_PUBLIC_USE MOCK_API` — `!== 'false'` means mocks ON.
- Per-slice overrides `EXPO_PUBLIC MOCK_{AUTH,PATIENTS,APPOINTMENTS,DASHBOARD,`

`PROFILE,TEAM,TREATMENTS}`` — flip one slice to real while the rest stay mocked (useful for incremental backend wiring).

- ****All** `.env` / `.env.example` files and all 3 EAS profiles set

`EXPO_PUBLIC_USE MOCK_API=false` **, so in every real build the mocks are OFF; they are a debug-only path. (Three slices are the exception — they have `*no*` real branch yet: `notifications-prefs`, `sessions`, `devices` — see `FEATURE_STATUS.md`).

Backend gotchas you will hit (full list in `IDENTA_WEB_REFERENCE.md §3`): list filters use nested `filter[search]` / `filter[category_id]` / `filter[status]` ; appointments send `reason` but read back `notes` ; dashboard snapshot returns camelCase + string decimals (remapped + `Number()` -coerced in `api/dashboard.ts`).

5. State, persistence & data flow

Store	File	Holds	Persistence
auth	<code>stores/auth.ts</code>	<code>user</code> , <code>tokens</code> (access/refresh + expiries), <code>isAuthenticated</code> , <code>isHydrating</code>	SecureStore <code>identa.session</code> (best-effort; legacy back-compat drops token-less blobs); sets Sentry user id
network	<code>stores/network.ts</code>	<code>isOnline</code> , <code>lastChangeAt</code>	subscribes to <code>NetInfo</code> at module load
theme	<code>stores/theme.ts</code>	<code>mode</code> (<code>light</code> <code>dark</code> <code>auto</code> , default <code>auto</code>), <code>effective</code>	AsyncStorage <code>@identa/theme_mode</code> ; follows <code>Appearance</code> for <code>auto</code>
ui	<code>stores/ui.ts</code>	global sheet open/close + appointment view-date requests	—

- **React Query** (`App.tsx`): `PersistQueryClientProvider` + `AsyncStorage` persister,

cache key `@identa/query-cache-v2` , `throttleTime 1500` , `maxAge 24h` , only `success` queries dehydrated. Defaults: `staleTime 5min` , `gcTime 24h` , `retry 1` , no refetch on focus/reconnect; mutations `retry: false` . `MutationCache.onError` shows a unified offline toast.

⚠ **Bump the cache key** -vN whenever a response mapper changes shape (e.g. a casing fix in `api/dashboard.ts`). Otherwise users hydrate stale rows — the classic symptom is dashboard cards showing a non-number.

- **Boot/hydration** is gated by `components/ui/SplashGate.tsx` : waits on auth

hydrate + theme hydrate + Inter fonts (fonts skipped on iOS), then renders.

6. Configuration, environment & builds

Environment (`.env` , `EAS env`)

Var	Purpose	Default
<code>EXPO_PUBLIC_API_URL</code>	backend base, must end in <code>/v1</code>	<code>http://10.0.2.2:8001/api/v1</code> (Android emulator → host localhost)
<code>EXPO_PUBLIC_USE MOCK_API</code>	master mock switch	<code>false</code> in all real configs
<code>EXPO_PUBLIC MOCK_*</code>	per-slice mock overrides	unset (= follow master)
<code>EXPO_PUBLIC_SENTRY_DSN</code>	crash reporting; empty ⇒ Sentry no-op	unset in dev

iOS simulator/devices can't use `10.0.2.2` — set `EXPO_PUBLIC_API_URL` to your machine's LAN IP (e.g. `http://192.168.x.x:8001/api/v1`) for iOS local testing.

`app.json`

- name `Identa`, slug `identa`, scheme `identa`, bundle/package `uz.identa.mobile`,

`runtimeVersion.policy = appVersion`.

- Plugins: expo-font, expo-secure-store, expo-image-picker, expo-notifications.
- iOS `infoPlist`: camera / photo-library / FaceID / notifications usage strings;

`associatedDomains` `identa.uz` + `app.identa.uz`.

- Android permissions incl. CAMERA / biometric / POST_NOTIFICATIONS (location blocked).
- ⚠ `extra.eas` is empty `{}` — no `EAS projectId` is wired, so Expo push-token

resolution may no-op until this is set (`eas init`). See `ROADMAP.md`.

`eas.json` **build profiles**

Profile	Output	Channel	Notes
<code>development</code>	dev-client APK + simulator	<code>development</code>	internal
<code>preview</code>	APK	<code>preview</code>	internal
<code>production</code>	Android app-bundle , <code>autoIncrement</code> , <code>appVersionSource: remote</code>	<code>production</code>	iOS too

All profiles set `EXPO_PUBLIC_API_URL=https://api.identa.uz/api/v1`, `EXPO_PUBLIC_USE MOCK_API=false`, and Sentry secrets. ⚠ The `submit` block has **placeholder Apple credentials** (`REPLACE_WITH_*`).

Build order is Android-first, then iOS (a product decision — Android ships first).

Commands

```
npm start | npm run android | npm run ios    # Expo dev server
npm run typecheck                             # tsc --noEmit — run before committing
npm test | npm run test:coverage              # jest
npm run test:e2e                             # maestro flows
npm run build:prod:android                   # EAS app-bundle (Android first)
npm run build:prod:ios                      # EAS iOS
npm run update:prod                          # EAS OTA update (production channel)
```

7. Cross-cutting concerns — `src/lib/`

- `sentry.ts` — `initSentry()` is a no-op without `EXPO_PUBLIC_SENTRY_DSN`;

`tracesSampleRate 0.1` , `sendDefaultPii: false` , scrubs auth/cookie/csrf headers and password/token/secret fields, strips auth-route request bodies from breadcrumbs. `wrapApp = Sentry.wrap` (root error boundary).

- `otaUpdates.ts` — `checkAndDownloadUpdate()` fired at boot (no-op in Expo

Go / dev); downloads in the background, applies on next cold start.

- `notifications.ts` — foreground handler, `getExpoPushToken()` , and

local appointment reminders (`scheduleAppointmentReminder` 30-min lead, `syncReminders` idempotent). Local reminders fully work; **remote push does not yet** (no device-registration backend — see `FEATURE_STATUS.md`).

- `offlineGuard.ts` — `OfflineError` , `requireOnline()` , `isOfflineError()` .
- `permissions.ts` — mirrors the web RBAC: admin/dentist = full; assistant via

`assistant_permissions` (`module.action`); a `read_only` subscription blocks manage. Use `canView` / `canManage` / `isSubscriptionReadOnly` — **never** check the role string directly.

8. Conventions (do these or things break subtly)

1. **Errors:** branch on `ApiError.kind` , never raw axios.
2. **Money** is UZS **integers** (no minor units). **Dates** YYYY-MM-DD , **times** HH:mm (24h).
3. **Writes require** `access_mode === 'full'` and the right assistant permission —

gate UI with `canManage(...)` ; the API layer also calls `requireOnline()` .

1. **Bump** `@identa/query-cache-vN` in `App.tsx` when a mapper's output shape changes.
2. **Default locale is** `ru` . New user-facing strings go in all three of

`src/i18n/translations` — `ru` , `uz` , `en` (no key is allowed to exist in only one).

1. Run `npm run typecheck` before every commit (`ci` = typecheck + coverage).
-

9. Testing surfaces (detail in `QA_VERIFICATION.md`)

- **Jest** (`src/**/*.__tests__` , ~23 files): api modules, lib helpers, stores

(95% floor), `api/client.ts` (80% floor), a few components. Global coverage floor is intentionally low; the high floors are where they matter (stores + client).

- **Maestro** E2E (`.maestro/flows/` , 10 flows) with tags ``smoke / auth / navigation / odontogram / treatment` — run via `npm run test:e2e[:smoke|:auth|:nav|:]`.

Identa Mobile — Feature Status

A screen-by-screen / feature-by-feature snapshot of what is **Done**, **Partial**, or **Missing**, with file-level evidence and parity notes vs the web app. Use this to know exactly what's left before calling the app shippable.

Verified against the codebase 2026-06-05 (3 independent read-throughs). Legend:  Done ·  Partial ·  Missing/stub.

TL;DR

The app is **further along than "70–75%" in the clinical core** — patients, treatments, appointments and clinical images are fully built and wired to the real API. The remaining work is concentrated in: **(a) auth extras** (Google sign-in, biometrics), **(b) three Settings sub-features with no backend yet** (notification prefs, active sessions, push-device registration), **(c) billing checkout**, and **(d) polish** (dark-mode rollout, locale persistence, EAS project wiring). None of the clinical CRUD paths are blocked.

Domain	Status	One-line
Auth (login/register/reset/change pw)		Solid; Google + biometric are stubs
Dashboard		Real KPIs; only the sparkline trend is synthetic
Patients (list/detail/CRUD/photo)		Complete
Odontogram		View-only — no standalone condition editing
Treatments		Full CRUD incl. tooth picker + images
Gallery / clinical images		Upload wired; scan-status + variant fallback handled
Appointments (week/day/CRUD/conflicts)		Complete
Payments / debts		Read view + quick-payment write; no invoice UI (intentional)
Settings — profile/hours/practice/pw/team/lang/theme		Complete, real APIs
Settings — notifications prefs		UI built, mock-only (no backend route)
Settings — active sessions		UI built, mock-only (no backend route)
Push notifications (remote delivery)		Token fetched but device registration is mock-only
Billing upgrade / in-app checkout		CTA is a <code>comingSoon</code> stub
Notification center / inbox (header bell)		<code>comingSoon</code> stub

Auth — (two stubs)

Done: Email/password login (sends `device_name`, splits the flattened `{user, tokens}`, lands the token before any authed request); register → auto login chain (register issues no tokens, by backend design); forgot/reset password with the full CSRF deep-link flow; change-password; email-verification resend + dashboard banner; logout (tolerant of CSRF failure, clears query cache + store); SecureStore session with backward-compatible hydrate; role/subscription gating on entry.

Missing / stub:

-  **"Continue with Google"** is a stub on both Login (`LoginScreen.tsx:217`) and

Register (`RegisterScreen.tsx:338`) → `toast.info(comingSoon)` . *(The web app just shipped a real Google link/connect flow — see web `27d3c1b` . Mobile parity is pending.)*

-  **Biometric / Face-ID unlock** — not implemented (no `expo-local-authentication`).
-  **"Remember me"** checkbox is decorative (state set but unused; the session is

always persisted).

Dashboard —

Done: snapshot query keyed to local "today"; camelCase→snake remap + `Number()` coercion (with the documented earlier bug fixed, `dashboard.ts:128-167`); KPIs for revenue, outstanding debt, today's appointments, next-appointment; swipe-to- complete/cancel with optimistic cache patch + reminder cancellation; permission- gated cards; offline/error/empty/after-hours states.

Caveat:  the finance **sparkline trend is synthetic** — `buildTrend()` fabricates a 7-day wobble because the backend returns no history series yet (`DashboardScreen.tsx:119-130`, `479-498`). The KPI numbers themselves are real. Either ship a real history endpoint or hide the sparkline before launch.

Patients —

Done: list with debounced search, category chips, archived + inactive(6-month) filters, pull-to-refresh, A–Z grouping, permission gating; detail/overview (vitals, contact, medical, 3 finance cards, upcoming appointments, treatment history, archive/restore/force-delete with type-to-confirm, scan-gated photo); full create/edit via `PatientFormSheet` (phones E.164-normalized, DOB wheel, category, allergies/meds/history, photo pick+upload+remove, client validation mirroring `StorePatientRequest`). `api/patients.ts` is complete (list/get/overview/create/update/archive/restore/forceDelete/photo/categories).

Minor gaps: list requests `per_page: 100` instead of true infinite scroll (`PatientListScreen.tsx:98`) — fine at clinic scale; no patient-level "quick payment" shortcut (the `/patients/{id}/quick-payments` endpoint is reached only through the treatment flow).

Odontogram — (view-only)

Done: 32-tooth chart (4 quadrants), condition colors from the summary endpoint, per-tooth treatment-count badges, summary cards, tooth-detail modal with per-tooth accounting + history.

Missing:  no way to create/edit a **standalone odontogram condition entry** — `listPatientOdontogram` (`odontogram.ts:21`) is never called and there are no write endpoints. Conditions change only indirectly via treatments. This is **by design** (treatments are the source of truth), but if the product wants direct charting it's unbuilt. Confirm the intended UX before treating it as a gap.

Treatments —

Done: full CRUD (`treatments.ts`): create/update/delete `POST/PUT/DELETE /patients/{id}/treatments` , single-fetch with images, payment via quick-payments. `TreatmentEditSheet.tsx` covers type, back-datable date (capped at today), an embedded **tooth picker** (odontogram), debt/paid amounts, 5000-char comment, photo upload, 422 field-error mapping, delete-with-confirm. Reachable from Patient Detail and Odontogram. Web parity met.

Gallery / clinical images —

Done: `ImagePickerSheet` (camera + library, Android modal-race workaround, selection cap), `PhotoGallery` , `LightboxViewer` . Upload wired as multipart with the critical `Content-Type: undefined` boundary fix (`treatments.ts:437`). `scan_status` handled (rejected hidden; `resolveTreatmentImageUrl` does preview→thumbnail→url fallback); patient photo gated on `pending / rejected` ; subscription `entry_image_limit` respected.

Appointments —

Done: week-grid + day-timeline views, segmented switch, week strip with busy-day dots, swipe navigation, "now" line, jump-to-today; conflict detection (`appointmentConflicts.ts` , excludes cancelled/no_show); create/edit/detail sheets (create sheet mounted globally, opened with patient/date prefill); optimistic status changes (blocks editing finalized appts), delete with reminder cancellation; correct `reason` → `notes` mapping (`appointments.ts:7-16`); list with `filter[date_from/to/ status]` .

Payments / debts —

Done: read-only debt/history view aggregating treatments by patient (paid/debt/ net totals, patient + history tabs, search, sorted by |balance|). The **write** path lives in `TreatmentDetailSheet` : record payment with amount validation + balance cap, **cash / card / bank transfer** selector, notes, via `recordQuickPayment` → `POST /patients/{id}/quick-payments` ; delete payment with cache invalidation. `read_only` gating is correct (`canManage(user, 'payments')`).

Notes:  no standalone invoice UI — **intentional** (mobile abstracts invoices away, documented `payments.ts:8-17`). `updatePayment` wrapper exists but has no edit UI (delete + re-record instead).

Settings — ● (UI complete; 3 sub-features mock-backed)

Done (real APIs): profile edit, working hours, practice info (all `/settings/profile`), password change, language, theme/appearance, **team/staff management with full CRUD** (`/team/assistants`, dentist-only gated), billing view of the real `user.subscription`, logout. All 11 sheets are wired.

Missing / mock-only (no backend route exists yet):

- 🚫 **Notification preferences** — `notifications.ts:8` hardcoded `USE MOCK = true`;

the sheet UI is real but persists nowhere.

- 🚫 **Active sessions** — `sessions.ts:11` hardcoded `USE MOCK = true` with seeded

fake sessions; "revoke" hits the mock.

- 🚫 **Push-device registration** — `devices.ts:8` hardcoded `USE MOCK = true`. The

Expo token is fetched at startup (`navigation/index.tsx:98-100`) but only ack'd to memory, so **remote push cannot be delivered**. *(Local appointment reminders DO work — `lib/notifications.ts`.)*

- 🚫 **Billing upgrade** CTA is a stub (`BillingSheet.tsx:84` → `comingSoon`) — no

in-app checkout.

- 🚫 **Notification center** behind the dashboard header bell is a stub

(`DashboardHeader.tsx:84` → `comingSoon`).

Parity with the web app

Present on web, **not** on mobile (decide per-product whether each is in scope):

- Audit logs · advanced analytics dashboard · standalone invoices.

Present on **both** and at parity: patients, appointments, treatments, payments (quick), odontogram (web has editing; mobile view-only), team/staff management, profile/practice settings, subscription/billing **view**.

Just shipped on web, **pending on mobile: Google account link/connect** (web `Settings` → `Connected Accounts`). The mobile FAQ/UX should follow once the mobile Google flow lands.

Identa Mobile — QA & Verification Plan

How to run the app, run the automated suites, and manually verify each feature before a release. Pair this with `FEATURE_STATUS.md` (what exists) and `ARCHITECTURE.md` (how it's wired).

Verified 2026-06-05.

1. Environments & how to run

Prerequisites

- Node + npm, the Expo tooling (`npx expo`), and either Android Studio (emulator) or Xcode (iOS simulator) — or a physical device with **Expo Go** (dev) / a dev build.

- For real-backend testing: the Laravel API running locally (`Identa web` backend) or pointed at staging/prod.

Point the app at a backend (`.env` → `EXPO_PUBLIC_API_URL` , must end in `/v1`)

Target	URL
Android emulator → host localhost	<code>http://10.0.2.2:8001/api/v1</code> (the default)
iOS simulator / physical device on LAN	<code>http://<your-LAN-IP>:8001/api/v1</code>
Staging/prod	<code>https://api.identa.uz/api/v1</code>

`10.0.2.2` is an **Android-emulator-only** alias for the host. iOS and physical devices must use the machine's LAN IP. This is the #1 "why won't it connect" trap.

Mock vs real

- `EXPO_PUBLIC_USE MOCK_API=true` → run with **no backend** (good for UI/nav QA).
- `=false` (the build default) → hit the real API.
- Flip one slice at a time with `EXPO_PUBLIC MOCK_{AUTH,PATIENTS,...}=false` while wiring a new endpoint.

Run

```
npm start           # dev server (press a = Android, i = iOS)
npm run android     # build+run Android
npm run ios         # build+run iOS
```

2. Automated tests (run these first — they're the cheap gate)

```
npm run typecheck   # tsc --noEmit — MUST be clean before any commit
npm test            # jest unit/integration
npm run test:coverage # jest + coverage (used by `ci`)
npm run ci          # typecheck + test:coverage (the gate)
npm run test:e2e     # all Maestro flows (needs a running device/emulator + app)
```

Jest (src/**/__tests__ , ~23 files)

Covers the **logic that must not regress**: `api/client.ts` (auth/refresh/error normalization — 80% floor), all `stores/*` (95% floor), `api` slices (`patients-search` , `patients-mutations` , `appointments` , `payments` , `treatments` , `team` , `odontogram`), and lib helpers (`format` , `groupPatients` , `permissions` , `offlineGuard` , `currentLocale` , `useDebounceValue`). Uses `axios-mock-adapter` . Global coverage floor is intentionally low; **don't chase the global number — keep the high floors on `stores/` and `api/client.ts` green.**

Maestro E2E (.maestro/flows/ , 10 flows, appld uz.identa.mobile)

Tagged for targeted runs:

Flow	Tags
00-smoke	smoke
01-login / 02-login-failure	smoke,auth / auth
03-tab-navigation / 04-patient-search	navigation
05-odontogram	navigation,odontogram
06-treatment-create / 07-treatment-delete	navigation,treatment
08-logout	auth,smoke
09-dashboard-smoke	smoke,navigation

```
npm run test:e2e:smoke      # fast confidence pass
npm run test:e2e:auth       # auth flows
npm run test:e2e:nav        # navigation
npm run test:e2e:odontogram # odontogram
npm run test:e2e:treatment  # treatment create/delete
```

E2E gaps to add (see `ROADMAP.md`): no flow for appointment create/edit, patient create/edit, payment recording, or settings. Add these before relying on Maestro as a release gate.

3. Manual QA checklist

Run this matrix on **both** a real Android build and an iOS build (clinical app → both platforms matter), against the **real backend**, with a **dentist** account and at least once with an **assistant** (limited permissions) and a **read-only** (expired) subscription.

Smoke (every build)

- ☐ Cold start: splash → lands on Login (logged out) or Dashboard (logged in).
- ☐ Login with valid creds; wrong creds shows a field/error toast, not a crash.
- ☐ Kill & relaunch → still logged in (SecureStore session survives).
- ☐ Airplane mode: cached lists still render; a write shows the offline toast;

back online → writes work.

- ☐ Logout returns to Login and clears data.

Auth

- ☐ Register → auto-login chain lands on Dashboard.
- ☐ Forgot password → email; reset deep link (`identa://reset-password?... /`

`https://identa.uz/reset-password`) opens `ResetPassword` prefilled.

- ☐ Change password from Settings.
- ☐ Email-verification banner appears for unverified users; "resend" works.
- ☐ ⚠ Google sign-in is a **stub** — expect a "coming soon" toast (not a bug yet).

Dashboard

- ☐ KPIs (revenue / outstanding debt / today's appointments) match the backend.
- ☐ Swipe an appointment row → complete/cancel updates optimistically and sticks.
- ☐ ⚠️ Sparkline is **synthetic** — don't validate its shape against real history.
- ☐ After-hours / empty / error / offline states render.

Patients

- ☐ Search (debounced), category filter, archived & inactive filters.
- ☐ Create patient (phone normalization, DOB wheel, category, photo upload).
- ☐ Edit patient; archive → restore → force-delete (type-to-confirm).
- ☐ Photo only shows when `photo_scan_status` is approved (upload a fresh one →

pending → appears after backend scan).

Appointments

- ☐ Week grid ↔ day timeline switch; week strip dots; swipe weeks; jump-to-today.
- ☐ Create appointment (patient + date prefill from other screens works).
- ☐ Conflict warning when overlapping a scheduled appointment.
- ☐ Status change (scheduled→completed/cancelled/no_show) + reminder cancels.
- ☐ Local reminder fires ~30 min before (leave the app, wait / set a near time).

Treatments / odontogram / images

- ☐ Open a patient → add treatment (type, back-dated date, tooth picker, amounts, comment, photos). Edit and delete.

- ☐ Record a payment on a treatment (cash/card/bank_transfer; amount capped at balance). Delete a payment; balances recompute.

- ☐ Odontogram chart colors + per-tooth counts match the summary; tooth-detail modal opens. ⚠️ No standalone condition editing (by design).

Payments

- ☐ Debt list totals (paid/debt/net) and per-patient balances are correct.
- ☐ As a **read-only** subscription: write controls are hidden/disabled

(record-payment, edit, create everywhere).

- ☐ As an **assistant** without `payments.manage` : same gating.

Settings

- ☐ Profile edit, working hours, practice info, password change → persist (real API).
- ☐ Team/staff: create assistant, set permissions, reset password, deactivate

(dentist-only; assistants don't see this section).

- ☐ Language switch (ru/uz/en) updates UI. ⚠️ **Known bug:** locale resets to `ru` on next cold start (not persisted) — see `ROADMAP.md`.

- ☐ Theme light/dark/auto. ⚠️ Dark mode is **partial** — expect some screens still light-styled.

- ☐ ⚠️ Notification prefs / Active sessions / Billing-upgrade are **mock or stub** — changes won't persist; don't file as bugs until those backends exist.

4. API contract verification

Because the backend has casing/naming traps, verify these explicitly when wiring or upgrading a slice (full contract in `IDENTA_WEB_REFERENCE.md`):

- ☐ List filters go out as nested `filter[search]` / `filter[status]` / `filter[`

`category_id]` / `filter[date_from|to]` / `filter[archived_only]`` — **not** flat.

- ☐ Appointment **writes send** `reason` , reads come back as `notes` .
- ☐ Dashboard snapshot arrives **camelCase** with string decimals →

remapped + `Number()` -coerced in `api/dashboard.ts` .

- ☐ Auth: login response is **flattened** `{...user, tokens}` ; `device_name` is the

Sanctum token name; **register returns no tokens** (must login after).

- ☐ `/auth/refresh` works **without** CSRF; the other auth routes **require** the

`GET /auth/csrf-token` bootstrap.

- ☐ Image URLs (`url` / `thumbnail_url` / `preview_url`) may be null while the backend

generates variants → `preview`→`thumbnail`→`url` fallback renders something.

5. Regression watch / release gotchas

- **Cache key:** if you change any `api/*` response mapper's output shape, bump

`@identia/query-cache-vN` in `App.tsx` . Skipping this hydrates stale rows from a previous version (classic symptom: a dashboard card shows a non-number).

- **EAS push:** `app.json` → `extra.eas` is empty (no `projectId`). Until `eas init`

wires it, expo push-token resolution may no-op — verify on a real build, not Expo Go.

- **Submit creds:** `eas.json` `submit` has `REPLACE_WITH_*` Apple placeholders — fill

before `npm run submit:ios` .

- **New strings** must exist in all three locales (`ru` / `uz` / `en`) — a missing key

silently renders the raw path.

- **Run on real hardware** before release: image picker, camera, push permission

prompts, haptics, and reanimated gestures behave differently from the simulator.

Identa Mobile — Continuation Roadmap

Prioritized remaining work to take the app from its current state to a shippable Android release (then iOS). Each item lists **why**, **where**, **backend dependency**, and a rough **effort** ($S \leq \frac{1}{2} \text{ day}$ · $M \approx 1\text{--}2 \text{ days}$ · $L \approx 3\text{+ days}$). Source of truth for "what exists" is `FEATURE_STATUS.md`.

Drafted 2026-06-05.

How to read this

The clinical core (patients, appointments, treatments, payments, images) is **done**. What's left is mostly: build/release plumbing, a few backend-dependent Settings features, auth extras, and polish. Items marked **[BE]** are blocked on a backend endpoint that does not exist yet — coordinate with the Laravel team.

P0 — Launch blockers (do before the first real build/release)

P0.1 — Wire the EAS project id · S

`app.json` → `extra.eas` is empty (`{}`). Without a `projectId`, EAS builds/OTA and **Expo push-token resolution no-op**. Run `eas init`, commit the generated id. *Verify on a real dev build, not Expo Go.* Files: `app.json`, `eas.json`.

P0.2 — Decide the dashboard sparkline · S

The finance trend line is **synthetic** (`DashboardScreen.tsx:119-130`, `479-498`). Shipping fabricated data in a medical/finance product is a credibility risk. Either: **(a)** hide the sparkline until a real history endpoint exists, or **(b) [BE]** add a `dashboard/revenue-series` endpoint and render real points. Recommend (a) now, (b) later.

P0.3 — Confirm release config · S

- `eas.json submit` has `REPLACE_WITH_*` Apple placeholders — fill before any iOS

`submit` (Android-first, so this can trail the Android launch).

- Confirm `production` profile env (`EXPO_PUBLIC_API_URL=https://api.identa.uz/api/v1`,

`USE MOCK API=false`, Sentry DSN) and that `runtimeVersion` policy matches your OTA plan.

P0.4 — Locale persistence · S

Language resets to `ru` on every cold start (it's never written to storage). A clinic that picks Uzbek will be annoyed daily. Persist the selected locale to `AsyncStorage` and rehydrate in `I18nProvider` (mirror the `theme` store pattern). Files: `src/i18n/index.tsx`, `src/lib/currentLocale.ts`.

P1 — Important (needed for a "complete" feeling app)

P1.1 — Remote push delivery · M · [BE]

Local appointment reminders work; **remote push does not** — `devices.ts:8` is `USE MOCK=true` and there's no `/devices/register` route, so the fetched Expo token goes nowhere. Needs: **[BE]** a device-registration endpoint, then flip `devices.ts` to real and confirm `navigation/index.tsx:98-100` posts the token. Depends on P0.1 (`projectId`).

P1.2 — Google sign-in on mobile · M

Stub on Login (`LoginScreen.tsx:217`) and Register (`RegisterScreen.tsx:338`). The **web app already shipped** the Google connect/link flow; mobile should reach parity. Implement with Expo's Google auth (`expo-auth-session` / native Google sign-in), exchange for a Sanctum token via the backend's Google endpoint. Confirm the backend's account-linking policy (existing-email accounts must link from Settings, per web `27d3c1b`) and mirror that UX/messaging on mobile.

P1.3 — Notification preferences backend + wire · S · [BE]

UI is built; `notifications.ts:8` is `USE MOCK=true`. Needs **[BE]** `/settings/notifications` GET/PUT, then remove the mock branch.

P1.4 — Active sessions backend + wire · S · [BE]

UI is built; `sessions.ts:11` is `USE MOCK=true`. Needs [BE] a sessions/activity list + revoke endpoint, then remove the mock. (Security-sensitive — pairs well with the web's session management.)

P1.5 — In-app billing / upgrade · L · [BE]

`BillingSheet.tsx:84` CTA is a `comingSoon` stub. Needs a checkout path (likely a PayX flow, possibly via WebView/redirect) coordinated with the backend billing service. Until then, the upgrade journey is web-only — make sure the copy points users to the web cabinet rather than dead-ending.

P2 — Polish & parity (post-launch or as time allows)

P2.1 — Dark-mode rollout · L

Theme infra exists (`theme.ts` , `useColors`), but most components import the static `colors` palette directly, so dark mode is only partially live despite `userInterfaceStyle: automatic`. Migrate components to `useColors()` screen-by-screen (it's tracked, ~dozens of files). Either finish it or hide the dark/auto option until it's complete.

P2.2 — Notification center / inbox · M

Dashboard header bell is a stub (`DashboardHeader.tsx:84`). Build an inbox screen (depends on P1.1 push + a backend notifications feed).

P2.3 — Odontogram direct condition editing · M

Currently view-only by design (conditions flow from treatments). If the product wants direct charting, wire `listPatientOdontogram` (`odontogram.ts:21` , unused) and add create/update/delete entry endpoints [BE] + UI. **Confirm the product intent first** — this may be intentionally out of scope.

P2.4 — Patient-level quick-payment shortcut · S

Add a "record payment" action on Patient Detail that opens the quick-payment flow directly (today it's only reachable through a treatment).

P2.5 — Real pagination · M

Lists fetch `per_page: 100` (patients) / `500` (appointments) instead of paginating. Fine at clinic scale; revisit if any clinic exceeds those counts (switch to infinite-scroll using the `PaginationMeta` the API already returns).

P2.6 — Expand E2E coverage · M

Maestro covers smoke/auth/nav/odontogram/treatment. Add flows for **appointment create/edit, patient create/edit, payment recording, settings** before treating Maestro as a release gate (`QA_VERIFICATION.md §2`).

P2.7 — Deep-link host parity · S

`app.json` declares `app.identa.uz` (associatedDomains / intent filters) but `linking.prefixes` only lists `identa.uz`. Align them if universal links on `app.identa.uz` are needed.

P2.8 — Decide on invoice UI · S

Mobile intentionally hides standalone invoices (`payments.ts:8-17`). Confirm that's the final product decision; if invoices are needed on mobile, scope a screen.

Recommended sequence

1. **P0 block** (projectId, sparkline decision, release config, locale persistence)

— small, unblocks a trustworthy Android build.

1. **P1.2 Google sign-in** + **P1.1 push** (both visible, expected features).
2. **P1.3 / P1.4** (notifications & sessions) once the backend endpoints land.
3. **P1.5 billing**, then **P2** polish (dark mode, inbox, pagination, E2E) as

capacity allows.

Backend coordination checklist (give this to the Laravel team)

- ☐ `/devices/register` (push tokens) — P1.1
- ☐ `/settings/notifications` GET/PUT — P1.3
- ☐ Active-sessions list + revoke — P1.4
- ☐ Mobile billing/checkout (PayX) entry — P1.5
- ☐ (Optional) dashboard revenue history series — P0.2(b)
- ☐ (Optional) odontogram condition CRUD — P2.3
- ☐ Confirm the Google account-linking contract for mobile — P1.2